Análise Arquitetural do Backend em C

Dissecação do Gerenciamento de Memória e Lógica de Estado

Minicurso de Programação em C

Resumo

Este documento apresenta uma dissecação linha a linha da implementação do arquivo `jogo.c`. O foco principal é desmistificar a interação entre ponteiros, estruturas de dados customizadas e a alocação dinâmica na memória *Heap*.

1 Estruturas de Dados e o Layout de Memória

Para que um sistema seja robusto, ele precisa modelar seus dados de forma clara. Iniciamos o código definindo os tipos de dados que representarão o universo do jogo.

```c
typedef enum {
    VAZIO = 0,
    JOGADOR_X = 1,
    JOGADOR_O = 2
} ValorCasa;
```

Por que fazer isso? A linguagem C não possui um tipo booleano rígido ou proteções fortes contra "números mágicos". Ao usar um `enum`, definimos um domínio fechado. Qualquer variável do tipo `ValorCasa` só deve receber esses três valores, tornando o código autodescritivo.

```c
typedef struct {
    ValorCasa *tabuleiro;
    int jogadas_realizadas;
} EstadoJogo;
```

A Estrutura de Estado: A `struct` funciona como um contêiner. Em vez de termos variáveis globais soltas, agrupamos o contexto do jogo. O campo `jogadas_realizadas` é um inteiro simples (ocupa 4 bytes). O campo `tabuleiro` é um **ponteiro**. Ele não guarda o tabuleiro em si, mas sim o endereço de memória onde o tabuleiro começará.

2 O Ponteiro de Estado Global e o Encapsulamento

```c
static EstadoJogo *jogo_atual = NULL;
```

Esta é a linha mais crítica da nossa arquitetura. Vamos dissecá-la:

- `EstadoJogo *jogo_atual`: Declaramos um ponteiro. Ele é apenas uma variável capaz de armazenar um endereço de memória (geralmente ocupa 8 bytes em sistemas de 64 bits). Ele foi tipado para apontar exclusivamente para regiões de memória formatadas como `EstadoJogo`.

- `= NULL`: Inicializamos o ponteiro apontando para o "nada"(endereço 0). Isso previne que ele aponte para lixo de memória residual.
- `static`: Este é o modificador de encapsulamento do C. Ele informa ao compilador que a variável `jogo_atual` só existe e só pode ser acessada de dentro do arquivo `jogo.c`. O C# (ou qualquer outro arquivo C) não consegue ler ou alterar esse ponteiro diretamente, sendo obrigado a usar nossas funções.

3 A Dinâmica da Alocação na Heap

A memória de um programa é dividida em regiões. Variáveis locais vivem na *Stack* (Pilha) e são destruídas ao fim da função. Para dados que precisam persistir (como o tabuleiro), usamos a *Heap*.

```
EXPORT void iniciar_jogo() {
    if (jogo_atual != NULL) {
        finalizar_jogo();
    }
    jogo_atual = (EstadoJogo*) malloc(sizeof(EstadoJogo));
    jogo_atual->tabuleiro = (ValorCasa*) malloc(9 * sizeof(
       ValorCasa));
    jogo_atual->jogadas_realizadas = 0;
    // ... loop de inicializa  o
}
```

A Dupla Indireção de Memória: Como o apontamento funciona na prática?

1. `malloc(sizeof(EstadoJogo))`: O Sistema Operacional encontra um espaço livre na Heap com tamanho suficiente para guardar um ponteiro e um inteiro. O endereço desse bloco é retornado e guardado em `jogo_atual`.
2. `malloc(9 * sizeof(ValorCasa))`: O SO encontra *outro* bloco na Heap, grande o suficiente para 9 *enums*. O endereço desse segundo bloco é guardado no campo `tabuleiro`, que vive dentro do primeiro bloco.

Temos, portanto, um ponteiro estático apontando para uma `struct` dinâmica, que contém um ponteiro apontando para um vetor dinâmico. Essa separação permite alocar peças de tamanhos variados sob demanda.

4 Acesso Físico via Transformação Linear

```
EXPORT int preenche_pos(int jogador, int linha, int coluna) {
    if (jogo_atual == NULL || linha < 0 || linha > 2 || coluna < 0
       || coluna > 2) return -1;

    int indice = (linha * 3) + coluna;
    ValorCasa *casa_alvo = &jogo_atual->tabuleiro[indice];

    if (*casa_alvo == VAZIO) {
        *casa_alvo = (ValorCasa)jogador;
        jogo_atual->jogadas_realizadas++;
```

```
        return 0;
    }
    return -1;
}
```

A Lógica da Jogada:

- **Validação:** O primeiro `if` é uma barreira de segurança. Impede acesso indevido à memória (Segmentation Fault) caso o C# envie coordenadas inválidas ou caso o jogo não tenha sido iniciado.
- **Mapeamento:** `indice = (linha * 3) + coluna`. Como a memória do computador é puramente linear (uma fita longa de endereços), transformamos as coordenadas matriciais (l, c) em um deslocamento unidimensional contíguo.
- **Desreferenciação (`*casa_alvo`):** Capturamos o endereço exato da casa calculada usando o operador `&`. Ao utilizar `*casa_alvo = jogador`, estamos instruindo a CPU a ir fisicamente até aquele endereço na Heap e sobrescrever o valor em binário com o identificador do jogador.